Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 8m 7s

Final repository review result

Το repository **είναι έτοιμο για τελική παράδοση στο μάθημα**, με βάση τα πραγματικά αρχεία του GitHub connector. Η υλοποίηση αντιστοιχεί στο αποδεκτό SaaS/microservices σχήμα του saasPlug και ενσωματώνει σωστά τη Phase C τροποποίηση με `scheduler-service` ως control-plane, όχι ως business orchestrator.

Το βασικό συμπέρασμα στηρίζεται σε τρία επίπεδα evidence:

1. **Canonical specs/docs**: τα final specs, architecture summary, verification report και submission checklist συμφωνούν μεταξύ τους.
2. **Runtime/config/source evidence**: `docker-compose.yml`, Nginx gateway, frontend API helper, services, scheduler code και tests επιβεβαιώνουν τα αρχιτεκτονικά constraints.
3. **Verification/GitHub evidence**: τα Phase B/C/D logs, issue statuses #17/#18 και test reports δείχνουν ολοκλήρωση χωρίς blockers.

What is confirmed

Area	Status	Evidence / notes
Course/project identity	Confirmed	Το `README.md` ορίζει το saasPlug ως NTUA ECE Technologies of S Services project για EV charging discovery, reservation, provider an billing και operator administration. README
Accepted architecture	Confirmed	Το final spec απαιτεί microservices-based Dockerized SaaS με web frontend, FastAPI services, separate PostgreSQL containers, Rabb και Nginx gateway. SPECIFICATIONS
Final service set after Phase C	Confirmed	Το `README.md` και `services/README.md` περιλαμβάνουν `scheduler service` μαζί με τα 10 backend services και `rabbitmq`. README README
One PostgreSQL container per stateful service	Confirmed	Το data-ownership table περιλαμβάνει κάθε service με δικό του DB container, συμπεριλαμβανομένου `scheduler-service -> schedule postgres`. README
Docker Compose runtime	Confirmed	Το `docker-compose.yml` περιλαμβάνει `scheduler-service` με `SCHEDULER_DATABASE_URL`, Docker-internal service URLs και depen σε `scheduler-postgres`, catalog, analytics, billing και reservation services. docker-compose
Scheduler database	Confirmed	Το Compose ορίζει `scheduler-postgres` με database `scheduler` κ dedicated volume. docker-compose
Frontend is gateway-only	Confirmed	Το frontend API helper απορρίπτει οποιοδήποτε path δεν αρχίζει απ `/api/`. api
Frontend routes and role separation	Confirmed	Το `App.tsx` έχει EV routes, provider-admin routes και operator rou role guards. App App App
EV map/search by viewport	Confirmed	Το `CatalogPage.tsx` χτίζει catalog request με bounds, εφαρμόζει debounce 350 ms και εμφανίζει viewport map controls. CatalogP CatalogPage
Provider services remain internal	Confirmed	Το Nginx config δεν εκθέτει red/green/blue provider routes· τα publi είναι auth/providers/catalog/status/reservations/analytics/billing/schedule nd. default
Service-owned `/internal/jobs/*` blocked at gateway	Confirmed	Το Nginx επιστρέφει `404` για `/api/catalog/internal/`, `/api/reservations/internal/`, `/api/analytics/internal/`, `/api/billing/internal/`. default default

Area	Status	Evidence / notes
Scheduler public API exposed correctly	Confirmed	Το Nginx εκθέτει μόνο `/api/scheduler/*` προς `scheduler-service` default
Scheduler is control-plane, not business orchestrator	Confirmed	Το scheduler README δηλώνει ότι κρατά job definitions/run history, service-owned internal endpoints με `X-Scheduler-Token`, δεν περιέ catalog/analytics/billing/reservation business logic και δεν διαβάζει ό DBs. README
Scheduler metadata model	Confirmed	Το `scheduler-service/app/models.py` έχει `scheduled_jobs` και `job_runs`, δηλαδή scheduler metadata/run history, όχι business ta models models
Scheduler operator-only APIs	Confirmed	Τα scheduler routes για jobs/runs/manual run καλούν `ensure_operator(principal)`. routes routes
Scheduler internal headers and safe summaries	Confirmed	Το internal scheduler client στέλνει `X-Scheduler-Token`, `X-Job-Run` `X-Correlation-Id` και κάνει redaction σε sensitive fragments. client
Service-owned scheduler endpoints	Confirmed	Catalog, billing και reservation έχουν internal job endpoints που απ scheduler token/job run id και εκτελούν service-owned logic. routes routes routes routes
RabbitMQ boundaries	Confirmed	Το spec ορίζει RabbitMQ για background propagation μόνο και όχι γ login, map search, point details, status lookup, reservation provider direct frontend actions. SPECIFICATIONS
Shared event package	Confirmed	`packages/events` περιέχει common event envelope, routing-key constants, topology, publisher/consumer/idempotency helpers και τ exchange `saasplug.events`. README
Event topology/idempotency	Confirmed	Το package README ορίζει canonical routing keys, queues/DLQs κ least-once/idempotent consumption. README README README
Provider Bearer auth	Confirmed	`.env.example` έχει `PROVIDER_AUTH_SCHEME=Bearer`, ενώ provider c χτίζει `Authorization: Bearer <apiKey>` όταν υπάρχει API key. config client
Official Postman Bearer auth source	Confirmed	Η official Postman collection για redPlug έχει collection-level bearer `{{apiKey}}`. redPlug.postman_collection

Area	Status	Evidence / notes
Provider YAML sources preserved	Confirmed	To redPlug YAML υπάρχει με provider base URL και endpoints `/po` `/point/{pointid}`, `/reserve/{pointid}`, `/reserve/{pointid}/{minutes}`. redPlug redPlug redP
Root gateway contract tests	Confirmed	Root tests verify public health routes, scheduler auth-required route 404 for internal job routes through gateway only. test_gateway_cor
Test breadth after Phase D	Confirmed	Final verification records 184 tests across backend services, fronte shared events and root gateway contracts. final-verification
Final docs	Confirmed	`submission-checklist.md` marks final architecture/runtime, all serv frontend coverage, final docs, diagrams and verification as Done. submission-checklist submission-checklist
Final PlantUML diagrams	Confirmed	`docs/diagrams/final/README.md` lists final use-case, component, deployment, data-model, activity and sequence diagrams. REA
Deployment diagram coherence	Confirmed	Deployment diagram includes frontend, gateway, all services, Rabb all PostgreSQL containers including `scheduler-postgres`, provider and internal dependencies. deployment deployment
AI logs completeness	Confirmed	`docs/ai/prompt-log.md` indexes runs `000`, `00B`, `001` - `020`, inclu Phase B/C/D and final closure. prompt-log
Issue #17 Phase D	Confirmed closed	GitHub issue #17 is `closed` with `state_reason: completed`. Phase D - Expand application te…
Issue #18 Phase C	Confirmed closed	GitHub issue #18 is `closed` with `state_reason: completed`. Phase C - Resolve final warning…
GitHub Project #2 closure	Recorded, not directly queryable here	The available connector exposed issue APIs, not GitHub Project AP Repo logs record Project #2 closure after validation. 020-comprehensive-test-expansio…

Warnings / non-blocking limitations

Warning	Evidence / impact
Full interactive Google OAuth browser callback was not run	Phase B verified non-interactive OAuth configuration and Google authorization URL generation, but not a full browser callback. This is acceptable for local MVP verification. 016-live-env-verification

Warning	Evidence / impact
Live reservation mutation smoke was intentionally skipped	Phase B skipped live reservation mutation because `RUN_LIVE_RESERVATION_SMOKE=true` was not set. This is a safety decision, not an implementation blocker. 016-live-env-verification
Browser visual QA / Figma QA not performed	Final docs record no callable browser/Figma source/tooling for that validation. This affects polish evidence, not functional submission readiness. submission-checklist
Transient PostgreSQL startup retry during Phase D	First full-stack startup saw transient catalog/status DB connection-refused errors; targeted restart succeeded and all containers became healthy. 019-phase-d-test-expansion
Local ignored `.env` existed during one audit	It was not read, modified, printed or committed. `.gitignore` excludes `.env`, and final Phase D check confirms `.env` is untracked. 015-finalization-project-audit 020-comprehensive-test-expansio…
Live provider functionality depends on real credentials outside repo	`.env.example` correctly uses placeholders; actual live credentials are intentionally outside version control. .env
RabbitMQ management port exposed for local dev	Compose exposes `15672:15672`, while frontend/source and gateway rules still prevent browser/RabbitMQ application coupling. This is a local development exposure, not a violation of “frontend must not call RabbitMQ.” docker-compose

Blockers

No blockers.

The repository evidence supports this: final verification concludes the project is ready for final course submission, subject only to documented MVP limitations; Phase D records 184 passing tests; and issues #17/#18 are closed as completed.

final-verification

Phase D - Expand application te… Phase C - Resolve final warning…

Final readiness

Ready for final NTUA course submission.

Η υλοποίηση είναι επαρκής ως local Dockerized MVP για το μάθημα: καλύπτει EV flows, provider admin flows, operator flows, provider API integration, microservices separation, one-DB-per-stateful-service, RabbitMQ event propagation, scheduler/control-plane amendment, final documentation, final UML sources, AI traceability και broad tests.

Το μόνο που θα πρόσεχα πριν από packaging: **μην συμπεριλάβεις κανένα ignored/local `.env` ή generated/runtime artifact**. Παράδωσε τα tracked repo artifacts και τα final docs/diagrams.

Final Codex full verification prompt

```
You are working in:

ntua-el21026/NTUA-saasPlug

Task type:

FINAL VERIFICATION ONLY - NO IMPLEMENTATION

Goal:

Run one final evidence-based verification pass for the completed saasPlug repositor

This is strictly a verification-only task. Do not implement new features, do not ch

Repository/content language rule:

All repository files, documentation, comments, README files, AI logs, Codex prompts

Canonical files to read first:

- AGENTS.md
- README.md
- .env.example
- .gitignore
- docker-compose.yml
- deploy/nginx/default.conf
- docs/specs/SPECIFICATIONS.md
- docs/specs/README.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/README.md
- docs/ai/prompt-log.md
- docs/ai/codex-runs/014-final-docs-uml.md
- docs/ai/codex-runs/015-finalization-project-audit.md
- docs/ai/codex-runs/016-live-env-verification.md
- docs/ai/codex-runs/017-scheduler-design.md
- docs/ai/codex-runs/018-scheduler-control-plane-implementation.md
- docs/ai/codex-runs/019-phase-d-test-expansion.md
- docs/ai/codex-runs/020-comprehensive-test-expansion-and-project-closure.md
- docs/ai/prompts/
- docs/diagrams/final/

Hard constraints:

- Do not implement new features.
- Do not change architecture.
```

```
- Do not change backend or frontend business logic.
- Do not add deployment, hosting, or Vercel.
- Do not create `.env`.
- Do not print secrets.
- Do not expose provider API keys, JWT secrets, Google secrets, RabbitMQ passwords,
- Do not commit, print, copy, or upload `.env`.
- Use `docker compose config --quiet` instead of printing interpolated Compose conf
- Do not create fallback project-management Markdown artifacts such as `docs/projec
- Do not close, reopen, or modify GitHub issues/projects unless explicitly instruct
- Clearly report unavailable tools instead of claiming success.
- Keep all repository content in English.

Verification scope:

1. Repository governance and secrets
   - Verify `.gitignore` excludes `.env` and secret-like local files while keeping
   - Verify `.env.example` is complete enough for the current runtime:
     - app/public base URL;
     - JWT;
     - Google OAuth;
     - RabbitMQ;
     - scheduler control-plane variables;
     - provider base URLs;
     - `PROVIDER_AUTH_SCHEME=Bearer`;
     - provider API key placeholders;
     - all service database URLs including `SCHEDULER_DATABASE_URL`.
   - Verify `.env` is not tracked using `git ls-files --error-unmatch .env`.
   - If a local `.env` exists, do not read it and do not print it. Only report that

2. Static architecture verification
   - Verify the final canonical service set exists:
     - auth-service
     - provider-registry-service
     - red-provider-service
     - green-provider-service
     - blue-provider-service
     - catalog-service
     - status-service
     - reservation-service
     - analytics-service
     - billing-service
     - scheduler-service
   - Verify Docker Compose includes:
     - frontend
     - nginx-gateway
     - rabbitmq
     - all backend services
     - one PostgreSQL container per stateful service:
       - auth-postgres
       - provider-registry-postgres
       - red-provider-postgres
       - green-provider-postgres
       - blue-provider-postgres
       - catalog-postgres
       - status-postgres
```

```
      - reservation-postgres
      - analytics-postgres
      - billing-postgres
      - scheduler-postgres
   - Verify no MongoDB runtime component exists.
   - Verify no dedicated orchestrator service exists.
   - Verify provider services are not exposed through Nginx public routes.
   - Verify Nginx exposes `/api/scheduler/*`.
   - Verify Nginx blocks service-owned internal job prefixes with 404:
     - `/api/catalog/internal/`
     - `/api/analytics/internal/`
     - `/api/billing/internal/`
     - `/api/reservations/internal/`

3. Frontend source verification
   - Verify frontend source calls only gateway-relative `/api/...` paths.
   - Verify there are no forbidden frontend calls to:
     - Docker-internal service hostnames;
     - direct service ports;
     - red/green/blue provider services;
     - external provider API URLs;
     - RabbitMQ or AMQP URLs;
     - service-owned `/internal/jobs/*` routes.
   - Verify EV, provider-admin, and operator routes still exist.
   - Verify operator scheduler UI/API usage remains under `/api/scheduler/*`.

4. Provider API/auth verification
   - Verify provider YAML and Postman collection files exist:
     - docs/assignment/providers/yaml/redPlug.yaml
     - docs/assignment/providers/yaml/greenPlug.yaml
     - docs/assignment/providers/yaml/bluePlug.yaml
     - docs/assignment/providers/postman_collections/redPlug.postman_collection.jso
     - docs/assignment/providers/postman_collections/greenPlug.postman_collection.j
     - docs/assignment/providers/postman_collections/bluePlug.postman_collection.js
   - Verify provider services send `Authorization: Bearer <apiKey>` by default thro
   - If real provider credentials and required network/VPN access are available, ru
     - list endpoint for redPlug, greenPlug, bluePlug;
     - one detail/status endpoint per provider when a safe ID is available.
   - Do not print API keys or raw response bodies.
   - Skip all live provider checks clearly if credentials or network/VPN are unavai
   - Skip live reservation mutations unless `RUN_LIVE_RESERVATION_SMOKE=true` is e

5. Tests and builds
   - Run frontend tests:
     - `docker compose run --build --rm --no-deps frontend npm test -- --run`
   - Run frontend production build:
     - `docker compose run --build --rm --no-deps frontend npm run build`
   - Run all backend service tests:
     - `docker compose run --build --rm --no-deps auth-service pytest -q`
     - `docker compose run --build --rm --no-deps provider-registry-service pytest
     - `docker compose run --build --rm --no-deps red-provider-service pytest -q`
     - `docker compose run --build --rm --no-deps green-provider-service pytest -q`
     - `docker compose run --build --rm --no-deps blue-provider-service pytest -q`
     - `docker compose run --build --rm --no-deps catalog-service pytest -q`
     - `docker compose run --build --rm --no-deps status-service pytest -q`
```

```
     - `docker compose run --build --rm --no-deps reservation-service pytest -q`
     - `docker compose run --build --rm --no-deps analytics-service pytest -q`
     - `docker compose run --build --rm --no-deps billing-service pytest -q`
     - `docker compose run --build --rm --no-deps scheduler-service pytest -q`
   - Run shared event package tests through a backend image:
     - `docker compose run --build --rm --no-deps analytics-service pytest -q /tmp/
   - If the exact shared-events test path differs in the current image, locate the

6. Docker Compose and runtime health
   - Run:
     - `docker compose config --quiet`
     - `docker compose up -d --build`
     - `docker compose ps`
   - Verify all containers become healthy.
   - If transient PostgreSQL startup connection-refused errors occur for services c

7. Gateway and API smoke checks
   - Verify public health routes:
     - `/health`
     - `/`
     - `/api/auth/health`
     - `/api/providers/health`
     - `/api/catalog/health`
     - `/api/status/health`
     - `/api/reservations/health`
     - `/api/analytics/health`
     - `/api/billing/health`
     - `/api/scheduler/health`
   - Verify authenticated EV smoke using safe demo credentials from `.env.example`
     - login;
     - `/api/auth/auth/me`;
     - a read-only EV route such as `/api/reservations/reservations/me`.
   - Verify authenticated provider-admin smoke:
     - login;
     - `/api/auth/auth/me`;
     - a read-only provider-scoped analytics or billing overview route.
   - Verify authenticated operator smoke:
     - login;
     - `/api/auth/auth/me`;
     - scheduler jobs listing through `/api/scheduler/jobs`.
   - Verify scheduler operator API smoke:
     - `/api/scheduler/health`;
     - `/api/scheduler/jobs`;
     - `/api/scheduler/runs`.
   - Verify service-owned internal job gateway paths return HTTP 404:
     - `/api/catalog/internal/jobs/catalog/freshness`
     - `/api/analytics/internal/jobs/freshness`
     - `/api/billing/internal/jobs/freshness`
     - `/api/reservations/internal/jobs/freshness`
   - Verify scheduler manual run smoke for the safe non-mutating job:
     - `billing.prepare-invoices`
   - The scheduler manual run must execute through the scheduler operator API and t
   - Do not issue invoices as part of this verification.

8. Root gateway contract tests
```

```
   - After the stack is running, run:
     - `python3 -m unittest discover tests -v`

9. Documentation and diagrams
   - Verify these final docs exist and are coherent:
     - README.md
     - docs/specs/SPECIFICATIONS.md
     - docs/architecture-summary.md
     - docs/final-verification.md
     - docs/submission-checklist.md
     - docs/README.md
     - docs/specs/README.md
     - docs/diagrams/final/README.md
     - docs/ai/prompt-log.md
   - Verify all final `.puml` files listed in `docs/diagrams/final/README.md` exist
   - If `plantuml` is available, run:
     - `plantuml -checkonly docs/diagrams/final/*.puml`
   - If PlantUML rendering/checking is unavailable, clearly report the missing tool

10. AI logging and final report
   - Create or update one final verification run report under:
     - `docs/ai/codex-runs/`
   - Use a new run ID/name such as:
     - `021-final-verification-only.md`
   - The report must include:
     - invocation prompt summary;
     - canonical files read;
     - exact verification commands run;
     - pass/fail/warning result for each check;
     - unavailable tools;
     - whether `.env` is untracked;
     - whether live provider checks were run or skipped;
     - whether live reservation mutation was skipped or explicitly enabled;
     - final blockers, if any;
     - final readiness conclusion.
   - Update `docs/ai/prompt-log.md` only if you create a new verification run entry
   - Update `docs/final-verification.md` and `docs/submission-checklist.md` only if
   - Do not rewrite final docs unnecessarily.

11. Cleanup
   - Run:
     - `docker compose down`
     - `docker compose ps -a`
   - Do not delete local volumes unless explicitly instructed.
   - Do not remove user local `.env`.

Final response format:

Return a concise final report with:

1. Result
2. Commands/checks run
3. Passed checks
4. Warnings / skipped checks
5. Blockers
```

```
6. Files changed
7. Final readiness

Remember: verification-only. No implementation, no architecture changes, no deploym
```

Day-to-day commands

Command	Πότε το χρησιμοποιείς	Τι κάνει
`cd /home/peppasmich/NTUA-saasPlug`	Πάντα πρώτο βήμα	Πηγαίνει στο root του project.
`docker compose --env-file .env up -d`	Καθημερινό άνοιγμα	Ξεκινά τα υπάρχοντα containers στο background, χωρίς rebuild.
`docker compose --env-file .env up -d --build`	Μετά από αλλαγές σε code/dependencies/Dockerfiles	Κάνει build όπου χρειάζεται και ξεκινά το stack.
`docker compose --env-file .env ps`	Μετά από start/restart	Δείχνει ποια containers τρέχουν και αν είναι healthy.
`docker compose --env-file .env logs --tail=100 <service>`	Debug συγκεκριμένου service	Δείχνει τα τελευταία logs. Π.χ. `scheduler-service`, `catalog-service`, `nginx-gateway`.
`docker compose --env-file .env logs -f <service>`	Live debug	Ακολουθεί logs σε πραγματικό χρόνο. Σταμάτα με `Ctrl+C`.
`docker compose --env-file .env restart <service>`	Όταν ένα service κόλλησε	Κάνει restart μόνο το συγκεκριμένο container.
`docker compose --env-file .env down`	Καθημερινό κλείσιμο	Σταματά και αφαιρεί containers/network, αλλά κρατά

Command	Πότε το χρησιμοποιείς	Τι κάνει
		volumes/data.
`docker compose --env-file .env config --quiet`	Πριν από σοβαρό run ή μετά από `.env` αλλαγή	Ελέγχει Compose config χωρίς να τυπώσει secrets.
`curl http://localhost:8080/health`	Γρήγορος έλεγχος gateway	Πρέπει να επιστρέψει healthy response.
`curl http://localhost:8080/api/scheduler/health`	Έλεγχος scheduler	Βλέπεις αν το scheduler-service απαντά μέσω gateway.
`python3 -m unittest discover tests -v`	Όταν το stack ήδη τρέχει	Τρέχει root gateway contract tests.

Browser URLs

URL	Χρήση
`http://localhost:8080/`	Κύρια εφαρμογή
`http://localhost:8080/login`	Login
`http://localhost:8080/map`	EV map/search
`http://localhost:8080/operator/dashboard`	Operator dashboard
`http://localhost:8080/operator/sync`	Scheduler/manual jobs/catalog imports
`http://localhost:8080/operator/system`	System health/run history
`http://localhost:15672/`	RabbitMQ management UI

Commands για tests

Bash

```bash
docker compose --env-file .env run --build --rm --no-deps frontend npm test -- --run
docker compose --env-file .env run --build --rm --no-deps frontend npm run build
```

Για ένα backend service:

Bash

```bash
docker compose --env-file .env run --build --rm --no-deps scheduler-service pytest -q
```

Για όλα τα services, τρέχεις αντίστοιχα με:

```
auth-service
provider-registry-service
red-provider-service
green-provider-service
blue-provider-service
catalog-service
status-service
reservation-service
analytics-service
billing-service
scheduler-service
```

Commands που να αποφεύγεις

Command	Γιατί να το αποφεύγεις
`docker compose down -v`	Διαγράφει volumes, άρα χάνεις local PostgreSQL/RabbitMQ data. Χρησιμοποίησέ το μόνο για πλήρες reset.
`docker compose config`	Τυπώνει interpolated config και μπορεί να εμφανίσει secrets από `.env`. Χρησιμοποίησε `config --quiet`.
`cat .env`	Τυπώνει secrets. Απόφυγέ το.
`git add .env`	Δεν πρέπει ποτέ να γίνει commit το real `.env`.
`docker system prune -a --volumes`	Πολύ επιθετικό cleanup. Μπορεί να σβήσει images, cache και volumes άλλων projects.
`docker compose --env-file .env down -v`	Ίδιο θέμα: διαγραφή βάσεων/data. Μόνο για fresh baseline.
`docker compose up` χωρίς `-d`	Δεν είναι λάθος, αλλά δεσμεύει το terminal. Για day-to-day προτίμησε `up -d`.
`curl` απευθείας σε internal service hostnames	Δεν είναι το σωστό contract. Χρησιμοποίησε gateway `http://localhost:8080/api/...`.
Άνοιγμα `localhost:5173`	Το frontend πρέπει να το βλέπεις μέσω Nginx στο `localhost:8080`.

Η πιο ασφαλής καθημερινή ρουτίνα

Άνοιγμα

```
Bash
```

```
cd /home/peppasmich/NTUA-saasPlug
docker compose --env-file .env up -d
docker compose --env-file .env ps
```

Άνοιξε:

```
http://localhost:8080/
```

Έλεγχος

Bash

```
curl http://localhost:8080/health
curl http://localhost:8080/api/scheduler/health
```

Logs όταν κάτι δεν πάει καλά

Bash

```
docker compose --env-file .env logs --tail=100 nginx-gateway
docker compose --env-file .env logs --tail=100 scheduler-service
docker compose --env-file .env logs --tail=100 catalog-service
```

Κλείσιμο

Bash

```
docker compose --env-file .env down
```

Κανόνας

Για καθημερινή χρήση:

```
up -d  -> ps -> browser -> down
```

Για αλλαγές σε κώδικα:

```
up -d --build -> ps -> test/check -> down
```

Για reset μόνο όταν το θέλεις συνειδητά:

```
down -v -> up -d --build
```

Για να κάνω νέο build πρέπει πρώτα να έχω κάνει down -v;

Σκέφτηκε για 5s

Όχι. **Για νέο build δεν χρειάζεται πρώτα** `down -v`.

Η σωστή εντολή είναι:

Bash

```
docker compose --env-file .env up -d --build
```

Αυτό:

- κάνει rebuild τα images που χρειάζονται,
- ξαναδημιουργεί/recreates τα containers που επηρεάζονται,
- κρατά τα PostgreSQL/RabbitMQ volumes,
- δεν διαγράφει δεδομένα.

Τι να χρησιμοποιείς

Περίπτωση	Εντολή	Σχόλιο
Καθημερινό άνοιγμα χωρίς αλλαγές	`docker compose --env-file .env up -d`	Γρήγορο start.
Έκανες αλλαγές σε code/Dockerfile/dependencies	`docker compose --env-file .env up -d --build`	Το κανονικό νέο build.
Θες rebuild ενός μόνο service	`docker compose --env-file .env up -d --build catalog-service`	Πιο γρήγορο από full stack rebuild.
Θες restart χωρίς rebuild	`docker compose --env-file .env restart catalog-service`	Για κολλημένο service.
Θες σταμάτημα χωρίς απώλεια δεδομένων	`docker compose --env-file .env down`	Safe daily shutdown.
Θες πλήρες reset βάσεων/data	`docker compose --env-file .env down -v`	Διαγράφει volumes. Μόνο συνειδητά.

Πότε χρειάζεται `down -v`

Μόνο όταν θέλεις να σβήσεις όλα τα local δεδομένα:

Bash

```
docker compose --env-file .env down -v
docker compose --env-file .env up -d --build
```

Αυτό διαγράφει:

- PostgreSQL data,
- RabbitMQ data,
- catalog imports,
- local reservations,
- invoices,
- analytics aggregates,
- scheduler run history.

Άρα **όχι για απλό build**.

Αν θέλεις πραγματικά “καθαρό” rebuild image

Χωρίς να σβήσεις DB volumes:

```bash
docker compose --env-file .env build --no-cache
docker compose --env-file .env up -d
```

Ή για ένα service:

```bash
docker compose --env-file .env build --no-cache frontend
docker compose --env-file .env up -d frontend nginx-gateway
```

Κανόνας

Για νέο build:

```bash
docker compose --env-file .env up -d --build
```

Για πλήρες data reset:

```bash
docker compose --env-file .env down -v
```

Μην τα συγχέεις. `--build` αφορά images/containers. `-v` αφορά data volumes.

Τι να κάνω τώρα που το ανοίγω για πρώτη φορά;

Σκέφτηκε για 12s

Πρώτη εκκίνηση - κάνε αυτά με τη σειρά

1. Άνοιξε Docker Desktop

Περίμενε να γράφει ότι το Docker engine τρέχει. Αν είσαι σε WSL, βεβαιώσου ότι το Docker Desktop έχει WSL integration ενεργό.

---

2. Πήγαινε στο root του repo

```bash
cd /home/peppasmich/NTUA-saasPlug
```

---

3. Έλεγξε ότι υπάρχει `.env`

```bash
ls -la .env .env.example
```

Θέλεις να υπάρχουν και τα δύο:

```
.env
.env.example
```

Μην κάνεις `cat .env`, γιατί περιέχει secrets.

---

4. Έλεγξε ότι το `.env` δεν είναι tracked

```bash
git ls-files --error-unmatch .env
```

Αν βγάλει error τύπου:

```
error: pathspec '.env' did not match any file(s) known to git
```

είναι **σωστό**. Σημαίνει ότι το `.env` δεν θα γίνει commit.

---

5. Έλεγξε Compose config χωρίς να τυπώσεις secrets

```bash
docker compose --env-file .env config --quiet
```

Αν δεν εμφανίσει τίποτα, είναι καλό.

Μην τρέχεις σκέτο:

```bash
docker compose config
```

γιατί μπορεί να εμφανίσει interpolated τιμές από `.env`.

---

6. Πρώτο build και start

Για πρώτη φορά, τρέξε:

```bash
docker compose --env-file .env up -d --build
```

Αυτό θα πάρει λίγη ώρα, γιατί θα χτίσει images για frontend και backend services.

---

7. Έλεγξε αν όλα έγιναν healthy

```bash
docker compose --env-file .env ps
```

Περίμενε να δεις τα containers σε `running` / `healthy`.

Αν κάποια services είναι ακόμα `starting`, περίμενε 30-60 δευτερόλεπτα και ξανατρέξε:

```bash
docker compose --env-file .env ps
```

Άνοιγμα στον browser

Άνοιξε:

```
http://localhost:8080/
```

Αυτό είναι το βασικό entrypoint της εφαρμογής. Το gateway εκτίθεται στο host port `8080`, ενώ το frontend και τα backend services μένουν εσωτερικά.

Login

Χρησιμοποίησε demo user.

Operator

```
operator@saasplug.local
```

EV user

```
evuser@saasplug.local
```

Provider admins

```
red-admin@saasplug.local
green-admin@saasplug.local
blue-admin@saasplug.local
```

Password: χρησιμοποίησε την τιμή του `DEMO_USER_PASSWORD` από το `.env`. Αν δεν το έχεις αλλάξει από το template, είναι:

```
change-me-demo-password
```

Πρώτη πρακτική ενέργεια μέσα στην εφαρμογή

Αφού μπεις ως operator:

```
http://localhost:8080/operator/sync
```

Τρέξε τα provider-specific catalog imports:

```
catalog.import.red
catalog.import.green
catalog.import.blue
```

Αυτό γεμίζει το catalog με live provider data. Είναι προτιμότερο από το bulk `catalog.import.all`, επειδή στο τελευταίο live verification το bulk import είχε timeout, ενώ τα τρία provider-specific jobs πέρασαν και γέμισαν σωστά το catalog.

Μετά πήγαινε ως EV user στο:

```
http://localhost:8080/map
```

και έλεγξε ότι εμφανίζονται charging points.

Γρήγορα health checks

Σε δεύτερο terminal:

```bash
curl http://localhost:8080/health
curl http://localhost:8080/api/auth/health
curl http://localhost:8080/api/catalog/health
curl http://localhost:8080/api/scheduler/health
```

Πρέπει να επιστρέφουν επιτυχή responses.

Αν κάτι δεν ανοίγει

Δες containers

```bash
docker compose --env-file .env ps
```

Δες logs gateway

```bash
docker compose --env-file .env logs --tail=100 nginx-gateway
```

Δες logs frontend

```bash
docker compose --env-file .env logs --tail=100 frontend
```

Δες logs scheduler

```bash
docker compose --env-file .env logs --tail=100 scheduler-service
```

Δες logs catalog

```bash
```

```
docker compose --env-file .env logs --tail=100 catalog-service
```

Αν ένα service κόλλησε:

Bash

```
docker compose --env-file .env restart <service-name>
```

Παράδειγμα:

Bash

```
docker compose --env-file .env restart catalog-service
```

Κλείσιμο όταν τελειώσεις

Για καθημερινό κλείσιμο:

Bash

```
docker compose --env-file .env down
```

Αυτό **δεν διαγράφει δεδομένα**.

Μην χρησιμοποιήσεις τώρα:

Bash

```
docker compose --env-file .env down -v
```

Το `-v` διαγράφει volumes, δηλαδή local βάσεις, imports, reservations, invoices, analytics, RabbitMQ data.

Η σημερινή σωστή σειρά

Bash

```
cd /home/peppasmich/NTUA-saasPlug
docker compose --env-file .env config --quiet
docker compose --env-file .env up -d --build
docker compose --env-file .env ps
```

Μετά άνοιξε:

```
http://localhost:8080/
```

Μπες ως operator, τρέξε:

```
catalog.import.red
catalog.import.green
catalog.import.blue
```

Μετά δοκίμασε ως EV user:

```
http://localhost:8080/map
```

Κλείσιμο:

Bash

```
docker compose --env-file .env down
```

Give me all the useful commands I need in a cell to copy. Dont overdo it.

Σκέφτηκε για 4s

Bash

```
# ============================================================
# saasPlug - Day-to-day Docker commands
# Run these from WSL, inside the repository root.
# ============================================================

# Go to project root
cd /home/peppasmich/NTUA-saasPlug

# Check that Docker Compose config is valid without printing .env secrets
docker compose --env-file .env config --quiet

# Start the app normally
docker compose --env-file .env up -d

# Start the app with rebuild after code/dependency/Dockerfile changes
docker compose --env-file .env up -d --build

# Check container status / health
docker compose --env-file .env ps

# Open in browser:
# Main app:              http://localhost:8080/
# Gateway health:        http://localhost:8080/health
# Scheduler health:      http://localhost:8080/api/scheduler/health
# RabbitMQ management:   http://localhost:15672/
```

```
# View recent logs for important services
docker compose --env-file .env logs --tail=100 nginx-gateway
docker compose --env-file .env logs --tail=100 frontend
docker compose --env-file .env logs --tail=100 scheduler-service
docker compose --env-file .env logs --tail=100 catalog-service

# Follow live logs for one service
docker compose --env-file .env logs -f scheduler-service

# Restart one service if it is stuck
docker compose --env-file .env restart scheduler-service

# Rebuild only one service
docker compose --env-file .env up -d --build frontend
docker compose --env-file .env up -d --build catalog-service

# Run frontend tests/build
docker compose --env-file .env run --build --rm --no-deps frontend npm test -- --run
docker compose --env-file .env run --build --rm --no-deps frontend npm run build

# Run one backend service test suite
docker compose --env-file .env run --build --rm --no-deps scheduler-service pytest -q

# Run root gateway contract tests after the stack is already running
python3 -m unittest discover tests -v

# Stop the app safely without deleting databases/volumes
docker compose --env-file .env down

# ============================================================
# Avoid in normal day-to-day use
# ============================================================

# Avoid: prints interpolated config and may expose .env-derived values
# docker compose config

# Avoid: prints secrets
# cat .env

# Avoid: never commit real secrets
# git add .env

# Avoid unless you intentionally want a full data reset:
# This deletes PostgreSQL/RabbitMQ volumes and local app data.
# docker compose --env-file .env down -v

# Avoid unless you intentionally want aggressive Docker cleanup:
# docker system prune -a --volumes
```